Alternate Printing (Even-Odd)

- **Create two threads: one prints even numbers, another prints odd numbers (1–100) in sequence.**

```
package Multi_thread;
public class Even_odd {
	private int c=1;
	private final int limit =10;
	private final Object lock = new Object();
	public void Odd() {
		synchronized (lock) {
			while(c<=limit) {
				if(c%2==0) {
					try {
		lock.wait();
	    } catch (InterruptedException e) {
	       Thread.currentThread().interrupt();
	    }
				}
				else {
	    System.out.println("Odd Thread: " + c);
	    c++;
	    lock.notify();
	  }
			}
		}
	}

	public void Even() {
		synchronized (lock) {
			while(c<=limit) {
				if(c%2!=0) {
					try {
		lock.wait();
	    } catch (InterruptedException e) {
	       Thread.currentThread().interrupt();
	    }
				}
				else {
	    System.out.println("Even Thread: " + c);
	    c++;
	    lock.notify();
	  }
			}
		}
	}

	public static void main(String[] args) {
		Even_odd obj=new Even_odd();
```

```
                Thread t1 = new Thread(() -> obj.Odd());
                Thread t2 = new Thread(() -> obj.Even());

                t1.start();
    t2.start();
        }
}
```

```
Odd Thread: 1
Even Thread: 2
Odd Thread: 3
Even Thread: 4
Odd Thread: 5
Even Thread: 6
Odd Thread: 7
Even Thread: 8
Odd Thread: 9
Even Thread: 10
```

Print A B C in Sequence

- **Three threads print A, B, C repeatedly in order (ABCABC…).**

```
package Multi_thread;
public class Sequence_abcabc implements Runnable {

        private static final Object lock = new Object();
  private static int status = 0;
  private final int threadId;
  private final char charToPrint;

  public Sequence_abcabc(int threadId, char charToPrint) {
    this.threadId = threadId;
    this.charToPrint = charToPrint;
  }

  public void run() {
    for (int i = 0; i < 10; i++) {
      synchronized (lock) {
        while (status != threadId) {
          try {
            lock.wait();
          } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
          }
        }

        System.out.print(charToPrint);
        status = (status + 1) % 3;
```

```
            lock.notifyAll();
        }
    }
  }
        public static void main(String[] args) {
                Thread t1 = new Thread(new Sequence_abcabc(0, 'A'));
    Thread t2 = new Thread(new Sequence_abcabc(1, 'B'));
    Thread t3 = new Thread(new Sequence_abcabc(2, 'C'));
    t1.start();
    t2.start();
    t3.start();
        }
}
ABCABCABCABCABCABCABCABCABCABC
```

Bank Account System

- **Multiple threads deposit and withdraw from the same account safely.**

```
package Multi_thread;
class Account {
  private double balance = 0;
  public synchronized void deposit(double amount) {
    System.out.println(Thread.currentThread().getName() + " is depositing: " + amount);
    balance += amount;
    System.out.println("New Balance after deposit: " + balance);
  }
  public synchronized void withdraw(double amount) {
    if (balance >= amount) {
      System.out.println(Thread.currentThread().getName() + " is withdrawing: " + amount);
      balance -= amount;
      System.out.println("Balance after withdrawal: " + balance);
    } else {
      System.out.println(Thread.currentThread().getName() + " failed! Insufficient funds for: " +
amount);
    }
  }
}
public class Bank_accnt {
                public static void main(String[] args) {
                        Account myAccount = new Account();
                        Thread t1 = new Thread(() -> {
                for (int i = 0; i < 3; i++) {
                  myAccount.deposit(100);
                }
              },"Depositor-1");

                        Thread t2 = new Thread(() -> {
                for (int i = 0; i < 3; i++) {
                  myAccount.withdraw(50);
```

```
                }
            }, "Withdrawer-1");

                        t1.start();
                        t2.start();
        }
}
```

```
Depositor-1 is depositing: 100.0
New Balance after deposit: 100.0
Depositor-1 is depositing: 100.0
New Balance after deposit: 200.0
Depositor-1 is depositing: 100.0
New Balance after deposit: 300.0
Withdrawer-1 is withdrawing: 50.0
Balance after withdrawal: 250.0
Withdrawer-1 is withdrawing: 50.0
Balance after withdrawal: 200.0
Withdrawer-1 is withdrawing: 50.0
Balance after withdrawal: 150.0
```

Online Food Delivery System

- **Multiple orders processed by limited delivery agents (thread pool).**

```
package Multi_thread;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
class Delivery implements Runnable {
  private String order;
  public Delivery(String name) {
     this.order = name;
  }
  public void run() {
     System.out.println(Thread.currentThread().getName() + " picked up: " + order);
     try {
        Thread.sleep(2000);
     } catch (InterruptedException e) {
        e.printStackTrace();
     }
     System.out.println(Thread.currentThread().getName() + " complete delivery of: " + order);
  }
}
public class Food_delivery {
        public static void main(String[] args) {
                ExecutorService t1 = Executors.newFixedThreadPool(3);
                for (int i = 1; i <= 5; i++) {
       String order = "Order #" + i;
       Delivery task = new Delivery(order);
```

```
        System.out.println("Placing " + order);
        t1.execute(task);
    }
                t1.shutdown();

     }
}
Placing Order #1
Placing Order #2
Placing Order #3
Placing Order #4
Placing Order #5
pool-1-thread-3 picked up: Order #3
pool-1-thread-2 picked up: Order #2
pool-1-thread-1 picked up: Order #1
pool-1-thread-2 complete delivery of: Order #2
pool-1-thread-3 complete delivery of: Order #3
pool-1-thread-3 picked up: Order #5
pool-1-thread-1 complete delivery of: Order #1
pool-1-thread-2 picked up: Order #4
pool-1-thread-3 complete delivery of: Order #5
pool-1-thread-2 complete delivery of: Order #4
```